Team Swift — Demo2

Contents

01. **The bug fix**

02. **Swift Light**

03. **Whats next**

Bug Fix: Problem & Objective

What is OpenStack Swift?

- A distributed object storage system that stores data across multiple servers.
- Uses a Ring to determine where data is stored and replicated.

The Problem

- RingBuilder.save() directly overwrote the existing .builder file.
- An interrupted save could corrupt the file and lose the previous valid configuration.

Our Objective

- Implement atomic save to prevent file corruption.
- Preserve the previous valid builder if saving fails, while ensuring successful saves produce a complete file.

Our Solution: Atomic Save

UPDATED RingBuilder.save()

```python
def save(self, builder_file):
    """Serialize this RingBuilder instance to disk.
    :param builder_file: path to builder file to save
    """
    # We want to be sure the builder id's are persistent, so this is the
    # only place where the id is assigned. Newly created instances of this
    # class, or instances loaded from legacy builder files that have no
    # persisted id, must be saved in order for an id to be assigned.
    id_persisted = True
    if self._id is None:
        id_persisted = False
        self._id = uuid.uuid4().hex

    temp_builder_filename = builder_file + (".new.%d" % os.getpid())

    try:
        with open(temp_builder_filename, 'wb') as fh:
            pickle.dump(self.to_dict(), fh, 2)
            fh.flush()
            os.fsync(fh.fileno())
        renamer(temp_builder_filename, builder_file, fsync=True)
    except Exception:
        try:
            os.unlink(temp_builder_filename)
        except OSError:
            pass
        if not id_persisted:
            self._id = None
        raise
```

How it works



Experimental Validation

Four experiments to validate RingBuilder.save()

Experiment

Exp1 Partial Serialization

What We Tested

Inject failure halfway through pickle.dump

Result

PASS: original builder unchanged and loadable

Experiment

Exp2 Process Interruption

What We Tested

Kill writer during serialization, before and after rename

Result

PASS: old file intact before replacement; complete new file after

Experiment

Exp3 Failure Cleanup

What We Tested

Inject dump/flush/fsync/rename exceptions

Result

PASS: old builder preserved, temporary files cleaned, errors propagated

Experiment

Exp4 Durability Ordering

What We Tested

Instrument successful save operations

Result

PASS: serialize → flush → file fsync → close → rename → directory fsync

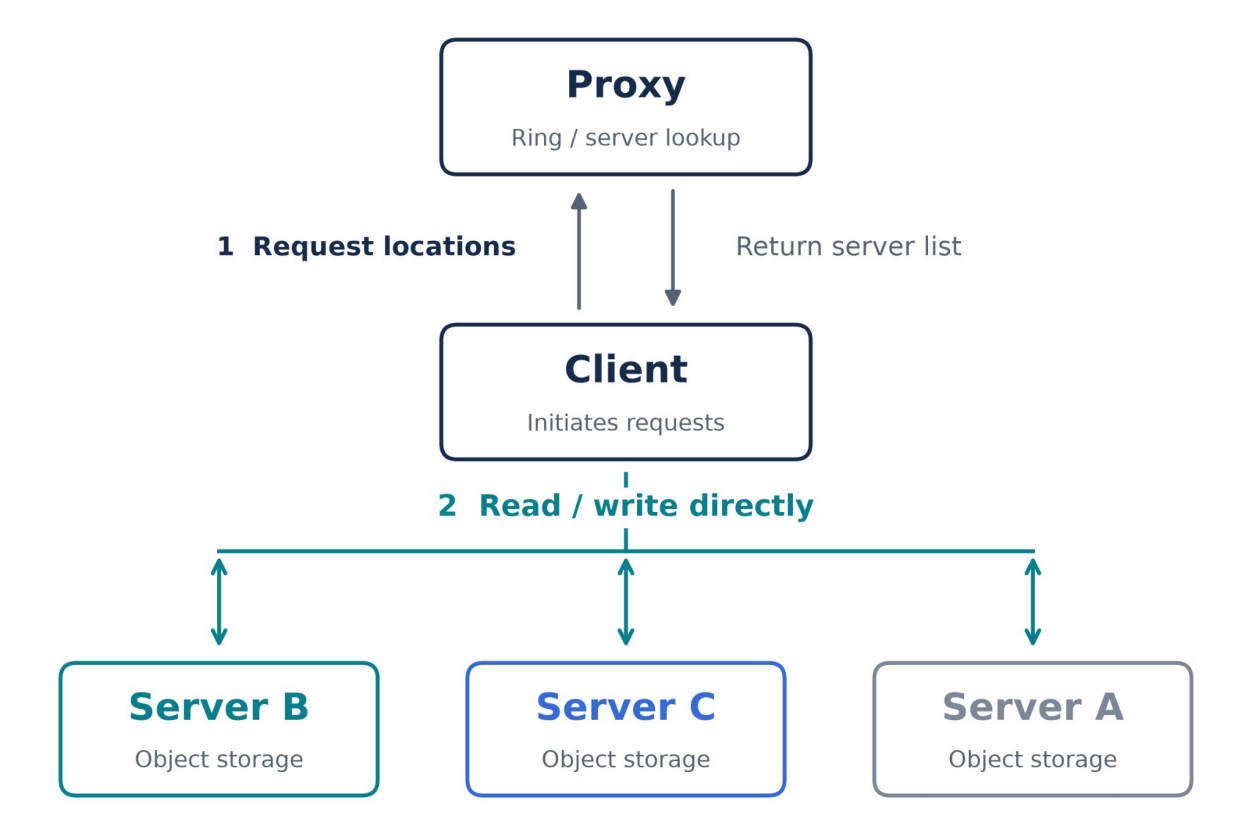
Swift Light

A lightweight distributed object store inspired by OpenStack Swift.

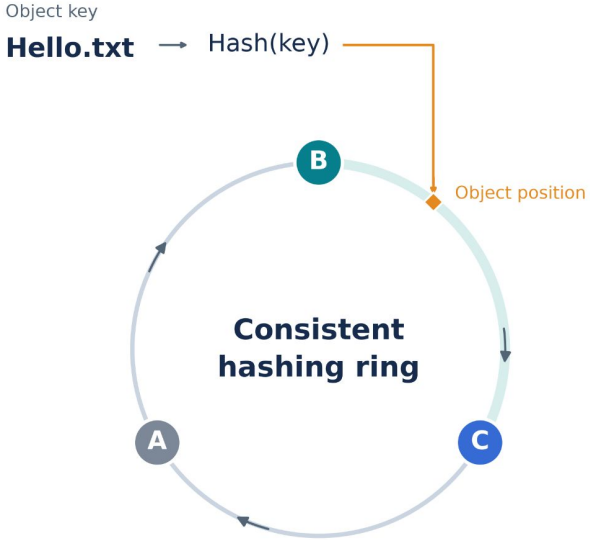
Consistent hashing determines object placement, with a proxy providing locations and clients reading and writing directly to storage nodes.

Atomic local saves and replicated writes protect object data, while fallback reads allow retrieval from another available replica.

Overview


Proxy
Ring / server lookup
1 Request locations
Return server list
Client
Initiates requests
2 Read / write directly
Server B
Object storage
Server C
Object storage
Server A
Object storage

Object Placement



Ordered server list

[B, C, A]

B **Primary**
First replica

C **Secondary**
Second replica

A **Substitute**
Used if a replica is unavailable

Simplified ring: one placement point per server shown.

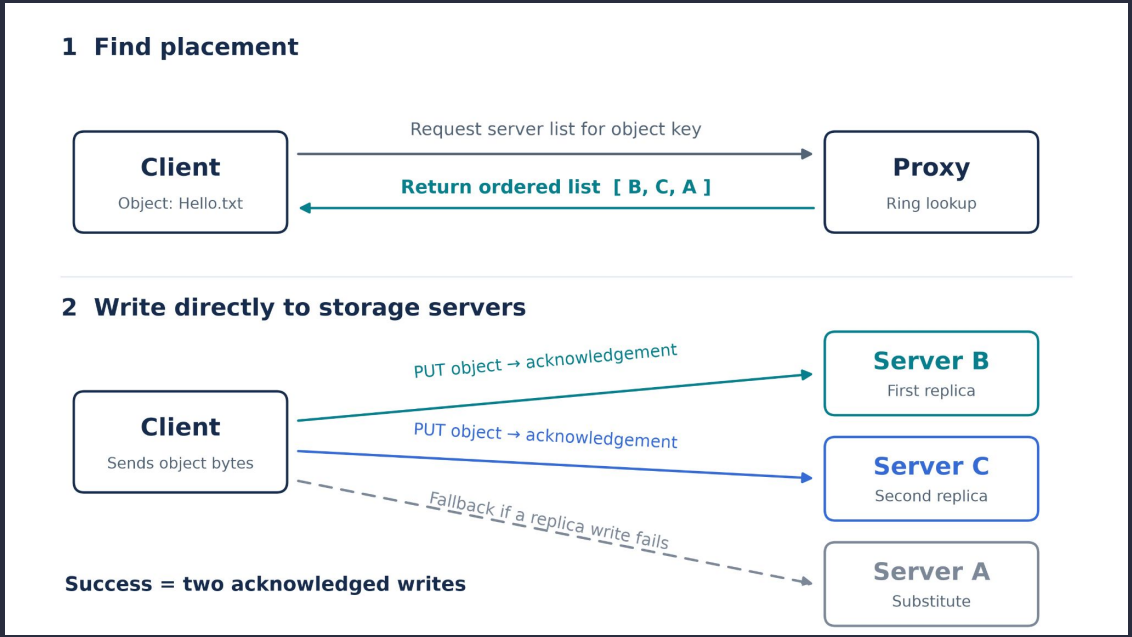
- Hash the object's key to a position on the ring.
- Use that position to select an ordered list of storage servers.
- Choose the first two servers as replicas; keep the remaining server as a substitute.

Write

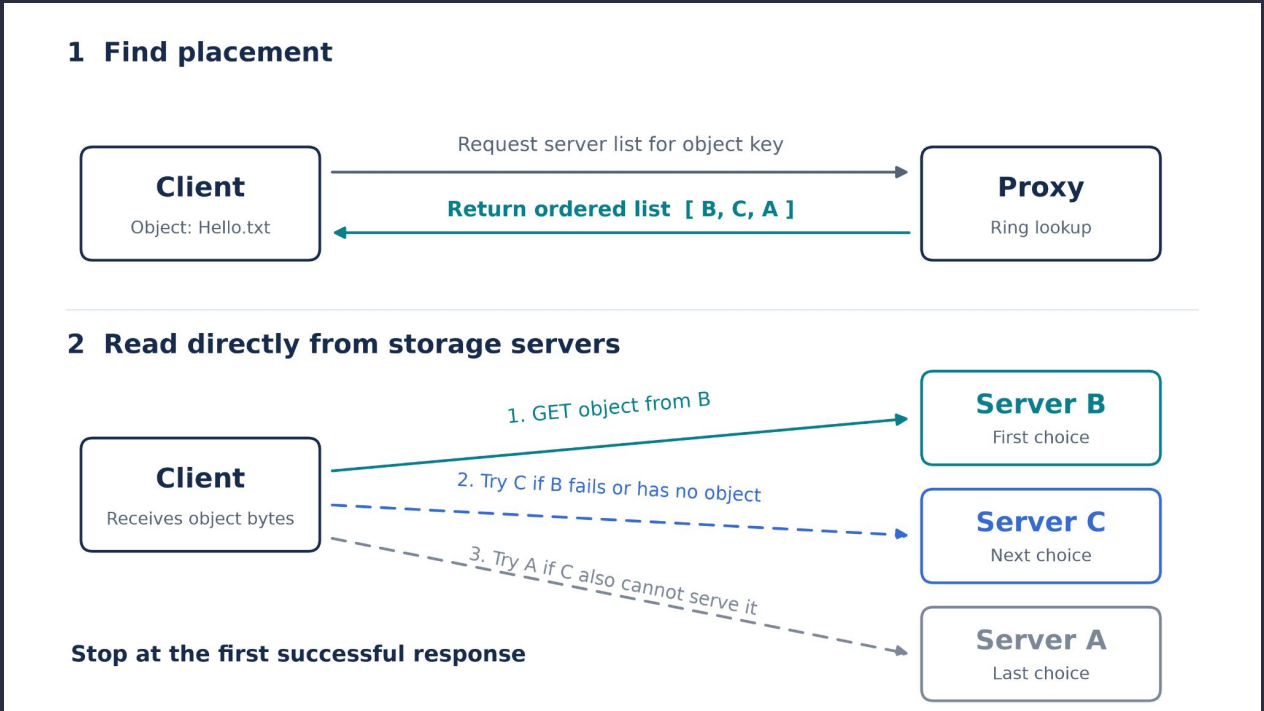
Writing an object:

1. The client asks the proxy for the object's ordered server list.
2. The client sends the object directly to B and C.
3. The client tries A if a replica server cannot accept the write.
4. Proxy checks for server availability on startup

An upload succeeds after two servers acknowledge the write.



Read

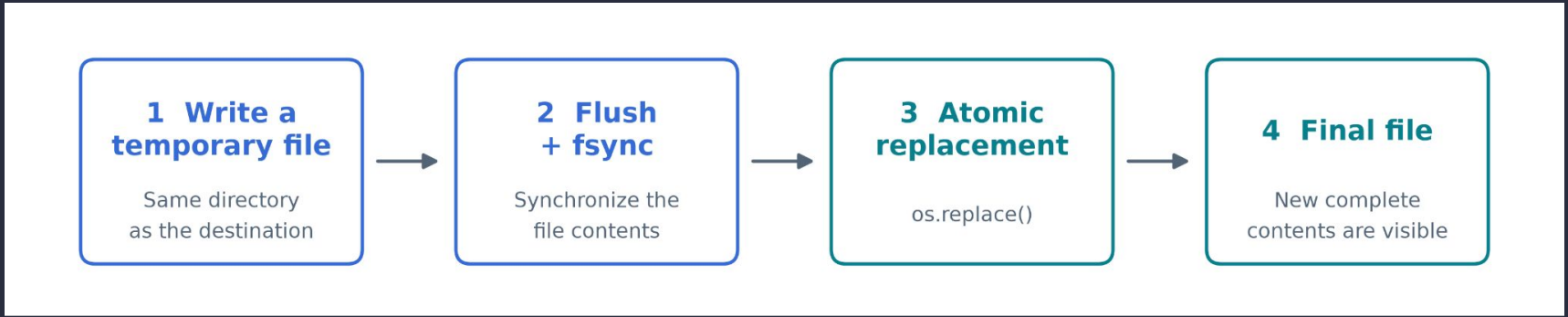


Reading an object: try replicas in order

1. The client asks the proxy for the object's server list.
2. The client requests the object directly from B.
3. If B is unavailable or does not have the object, try C, then A.

The first successful response supplies the object.

Atomic Save



Atomic writes: publish only complete files

1. **Write a temporary file** in the destination directory.
2. **Flush and synchronize** the file's contents to storage.
3. **Atomically replace** the destination with the completed file.

Readers see the previous complete file or the new complete file—never a partially written replacement.

Whats Next?

Why Swift Light, and what comes next?

- **Simplify experimentation:** A small system that preserves ring-based placement, replication, and atomic file writes.
- **Current:** verify uploads, replicas, and downloads.
- **Planned:** measure atomic-write overhead

Thank you